

Git & GitHub

Complete Journey - MediBook Phase 1

COMPLETE GIT WORKFLOW DOCUMENTATION

Every command explained letter by letter
Real mistakes I hit · Real fixes applied
From zero to GitHub push - the entire journey

What This Guide Contains

Every Git command broken down (flag by flag)

PowerShell vs CMD differences explained

The embedded git repository disaster I faced

How I debugged it (step by step)

Personal Access Token setup walkthrough

GitHub authentication errors and fixes

Branch strategy (main vs master vs feature)

Conventional commit message format

Professional Git workflow used at Google/Netflix

All 11 commands I ran to push Phase 1

Cheat sheet for future reference

Recovery strategies when things go wrong

Repository: github.com/VIJAYKUMAR-G-CHATTARAKI/MediBook

Tools: Git for Windows · PowerShell · GitHub PAT · IntelliJ IDEA

Table of Contents

PART I GIT FUNDAMENTALS

1. What is Git? Why Do We Need It?
2. Git vs GitHub - The Critical Difference
3. How Git Tracks Your Files (3 Areas)
4. Anatomy of a Git Command

PART II POWERSHELL VS CMD

5. Why This Matters on Windows
6. Command Equivalents Cheat Sheet
7. Common Mistakes from Mixing Them

PART III INITIAL SETUP

8. Installing Git on Windows
9. Configuring Git (One-Time Setup)
10. Creating .gitignore (Most Critical Step!)

PART IV MY ACTUAL JOURNEY (REAL COMMANDS)

11. Command 1: First git init attempt
12. Command 2: Discovering the Embedded Git Issue
13. Command 3: Locating the Rogue .git Folder
14. Command 4: Killing the Inner .git
15. Command 5: Verification
16. Command 6: Resetting the Stage
17. Command 7: Fresh git add
18. Command 8: Verifying Staging
19. Command 9: First Commit
20. Command 10: Connecting to GitHub
21. Command 11: The Push
22. Command 12: Creating Feature Branch

PART V DEEP DIVE - EVERY FLAG EXPLAINED

23. git init - Initialize a Repository
24. git add - Stage Files
25. git status - Check State
26. git commit - Save a Snapshot
27. git remote - Connect to GitHub
28. git push - Upload to GitHub
29. git rm - Remove Files
30. git checkout - Switch Branches
31. git branch - Manage Branches
32. git log - View History

PART VI GITHUB SETUP

33. Creating a GitHub Account
34. Creating Your First Repository
35. Personal Access Token (PAT) Setup
36. Why Password Auth Was Removed

PART VII PROFESSIONAL WORKFLOW

- 37. Feature Branch Strategy
- 38. Conventional Commit Messages
- 39. Pull Request Workflow
- 40. Commit Rhythm - When to Commit

PART VIII REFERENCE

- 41. Mistakes I Made and What I Learned
- 42. Quick Reference Cheat Sheet
- 43. Recovery from Common Disasters
- 44. Interview Talking Points
- 45. Glossary of Git Terms

PART I · GIT FUNDAMENTALS

1. What is Git? Why Do We Need It?

Git is a version control system. Think of it as "infinite undo" for your code, combined with the ability to collaborate with others without overwriting each other's work.

The Problem Git Solves

Why Git Exists

WITHOUT GIT (the bad old days):

```
my_project_v1/  
my_project_v2/  
my_project_v2_final/  
my_project_v2_final_FINAL/  
my_project_v2_final_FINAL_use_this_one/  
my_project_backup_2024_01_15/
```

Problems:

- Can't remember what changed between versions
- Can't easily go back to a specific state
- Two people editing = chaos
- Disk space wasted on copies
- No history of WHY changes were made

WITH GIT:

```
my_project/  <- ONE folder, with magical .git/ inside
```

Benefits:

- Every change is tracked with timestamp + author + message
- Easy to go back to ANY previous state
- Multiple people work on same files safely
- Tiny storage (only stores differences)
- Branch off to try ideas, abandon if bad

Real-World Use Cases

- **Solo developer:** Track your own work, revert mistakes, experiment safely
- **Team development:** Multiple devs work on same code without conflicts
- **Open source:** Anyone can contribute, you can review their changes
- **Deployment:** Production servers pull specific Git versions to deploy
- **Code review:** See what changed before merging code

Why this matters » Without Git, professional software development is impossible. Every company on Earth uses Git or something similar. Learning Git isn't optional - it's as fundamental as learning to type.

2. Git vs GitHub - The Critical Difference

Beginners confuse these constantly. They're related but completely different.

Aspect	Git	GitHub
What it is	A program (software)	A website (service)
Where it runs	Your computer	github.com servers
Cost	Free, open-source	Free for public, paid for private features
Created by	Linus Torvalds (2005)	Acquired by Microsoft (2018)
Purpose	Track changes locally	Host repos online + collaboration
Internet needed?	No - works offline	Yes - for push/pull
Alternatives	Mercurial, SVN	GitLab, Bitbucket, Codeberg

The Analogy

Easy Analogy

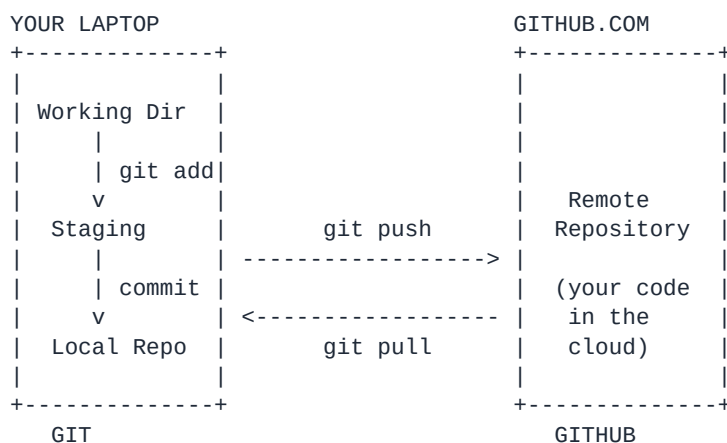
Git = Microsoft Word (the program on your computer)
GitHub = Google Drive (the cloud where you upload Word docs)

You use Word to write documents.
You upload to Google Drive to share with others.

Same here:
You use Git to track your code.
You push to GitHub to share with others.

How They Work Together

Git + GitHub workflow



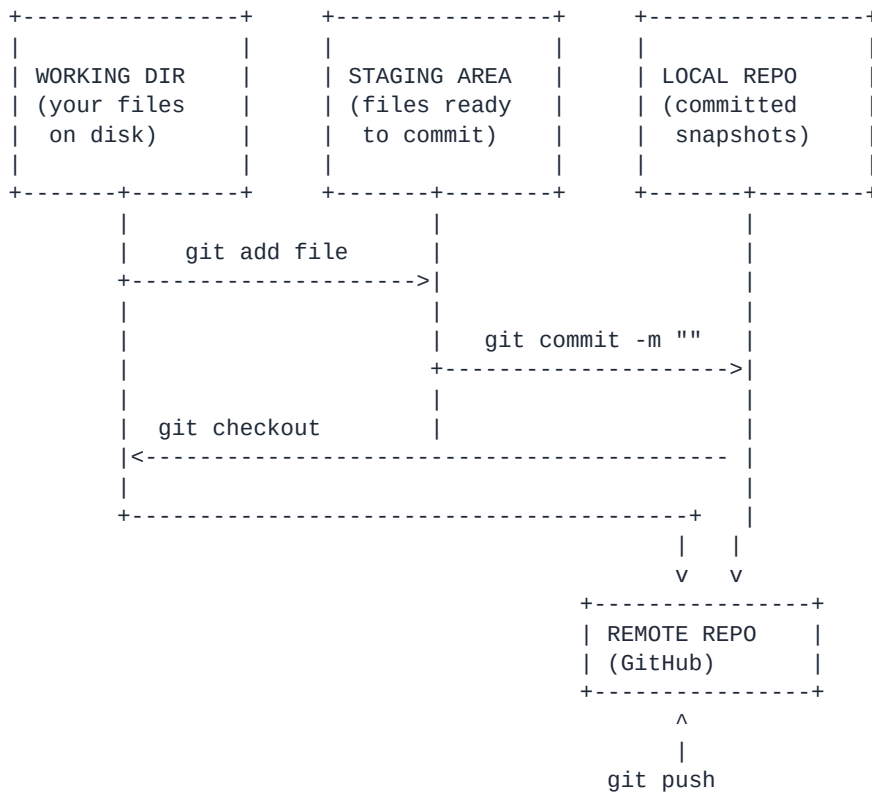
TIP » You can use Git **WITHOUT** GitHub (just track changes locally). But GitHub requires Git. Git is the engine; GitHub is the cloud hosting platform.

3. How Git Tracks Your Files (3 Areas)

This is the most important concept in Git. Most beginner confusion comes from not understanding these 3 areas.

The 3+1 Areas

THREE AREAS IN GIT



Working Directory

Your actual files on disk. When you edit a Java file in IntelliJ, you're editing the working directory. Git can see these files but they're not yet tracked by it.

Staging Area (also called "Index")

A waiting room. You move files here with **git add** when you've decided "these changes belong together as one commit". Think of it like a shopping cart - you add items before checkout.

Local Repository

The **.git/** folder. Contains the full history of all commits. When you **git commit**, files move from staging to here as a permanent snapshot.

Remote Repository (GitHub)

A copy of your local repo hosted on GitHub. When you **git push**, your local commits get uploaded to GitHub so others (and you on other devices) can access them.

Why this matters » Understanding these areas is critical. "git add" doesn't commit anything. "git commit" doesn't push anything. Each command moves files from one area to the next. When something goes

wrong, knowing WHICH area has the problem is half the battle.

4. Anatomy of a Git Command

Every Git command has a structure. Understanding it makes new commands easier to learn.

Git command structure

```
git    commit    -m    "Add user login"
|         |         |         |
|         |         |         +-- ARGUMENT: value for the option
|         |         +-----+ OPTION/FLAG: modifies behavior
|         +-----+ SUBCOMMAND: what to do
+-----+ PROGRAM NAME: invoke Git
```

More examples:

```
git    add    .
|         |         |
|         |         +-- ARGUMENT: . means "current folder and all subfolders"
|         +-----+ SUBCOMMAND: add to staging
+-----+ PROGRAM NAME
```

```
git    push    -u    origin    main
|         |         |         |         |
|         |         |         |         +-- ARG: branch name
|         |         |         +-----+ ARG: remote name
|         |         +-----+ FLAG: -u sets upstream tracking
|         +-----+ SUBCOMMAND
+-----+ PROGRAM NAME
```

```
git    rm    -r    --cached    .
|         |         |         |         |
|         |         |         |         +-- ARG: path (. = everything)
|         |         |         +-----+ LONG FLAG: --cached means "from index only"
|         |         +-----+ SHORT FLAG: -r means recursive
|         +-----+ SUBCOMMAND
+-----+ PROGRAM
```

Short vs Long Flags

Short Form	Long Form	Meaning
-m	--message	Specify commit message
-u	--set-upstream	Set tracking branch
-r	--recursive	Apply to subdirectories
-f	--force	Force the operation
-v	--verbose	Show detailed output
-b	--branch	Specify branch name

TIP » Short flags are quicker to type. Long flags are more readable in scripts. Both work the same way. Use short for terminal, long for scripts.

PART II · POWERSHELL VS CMD

5. Why This Matters on Windows

Windows has TWO different command-line interfaces: **Command Prompt (cmd.exe)** and **PowerShell**. They look similar but use different command syntax. Git commands work in both, but file-system commands like "delete this folder" are different!

MY MISTAKE » I learned this the hard way. I tried to run `rmdir /s /q .git` in PowerShell and got: "**A positional parameter cannot be found that accepts argument '/q'.**"

Reason: `rmdir` in PowerShell is an alias for **Remove-Item**, which uses different flags. The slash-based flags like `/s` and `/q` are CMD-style only.

Reason: **rmdir** in PowerShell is an alias for **Remove-Item**, which uses different flags. The slash-based flags like /s and /q are CMD-style only.

How to Tell Which One You're In

Visual difference

```
COMMAND PROMPT (cmd.exe):  
    Prompt looks like: C:\projects\medibook-service>  
                      ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
```

POWERSHELL:
 Prompt looks like: PS C:\projects\medibook-service>
 ^^ ^^^
 |
 +-- "PS" prefix is the giveaway!

Why PowerShell Exists

- Microsoft built it as a modern replacement for CMD
- More powerful - object-oriented, not just text
- Standard on modern Windows (10, 11)
- Used in DevOps automation, Azure scripts
- IntelliJ's built-in terminal defaults to PowerShell on Windows

WARNING » If you copy-paste commands from a Linux/Mac tutorial, they may fail in PowerShell. Always check which shell the tutorial assumes.

6. Command Equivalents Cheat Sheet

Same tasks, different syntax. Bookmark this page!

Task	CMD	PowerShell
Delete folder	rmdir /s /q myfolder	Remove-Item myfolder -Recurse -Force
Delete file	del myfile.txt	Remove-Item myfile.txt
Change directory	cd path	cd path (same)
List files	dir	ls or dir or Get-ChildItem
List hidden files	dir /a	Get-ChildItem -Force
Find files recursively	dir /s /b *.java	Get-ChildItem -Recurse -Filter *.java
Search text in files	findstr "text" file.txt	Select-String "text" file.txt
Check if file exists	if exist file.txt	Test-Path file.txt
Set env variable	set VAR=value	\$env:VAR = "value"
Make new folder	mkdir newfolder	mkdir newfolder (same)
Copy file	copy src.txt dest.txt	Copy-Item src.txt dest.txt
Move/rename file	move old.txt new.txt	Move-Item old.txt new.txt
Print to screen	echo Hello	Write-Host Hello or echo Hello
Clear screen	cls	cls or Clear-Host
Show env variables	set	Get-ChildItem env:
Run external program	program.exe args	& program.exe args
Pipe to count lines	command find /c /v ""	command Measure-Object -Line

TIP » If you want CMD commands inside PowerShell, just type **cmd** to switch into CMD mode. Type **exit** to go back to PowerShell.

7. Common Mistakes from Mixing Them

MY MISTAKE » Mistake #1 I made: Used `rmdir /s /q .git` in PowerShell.

Result: PowerShell tried to run Remove-Item with /s and /q as positional args - error.

Fix: `Remove-Item .git -Recurse -Force`

MY MISTAKE » Mistake #2 (common): Using `set VAR=value` in PowerShell.

Result: Nothing happens - PowerShell doesn't recognize CMD syntax.

Fix: `$env:VAR = "value"`

MY MISTAKE » Mistake #3 (common): Using `if exist file.txt` in PowerShell.

Result: Syntax error.

Fix: `if (Test-Path file.txt)`

My Recovery Strategy

Troubleshooting flow

WHEN A COMMAND FAILS IN POWERSHELL:

1. Look at the error message carefully
"A positional parameter cannot be found" = wrong syntax for PowerShell
2. Check which shell you're in (PS prefix or not)
3. If you got the command from a Linux/Mac/CMD tutorial:
 - Convert to PowerShell equivalent (see cheat sheet)
 - OR drop into CMD by typing 'cmd' then run the original
 - OR install Git Bash for Linux-style commands
4. Git commands always work the same in any shell.
Only the OS commands (rm, mkdir, etc.) differ.

WHAT I LEARNED » After hitting the `rmdir` mistake, I now ALWAYS pause when a Windows command fails. The error message tells you which shell rejected it. PowerShell error language is different from CMD error language - learn to recognize both.

PART III · INITIAL SETUP

8. Installing Git on Windows

Git is not pre-installed on Windows. You have to download it.

Step-by-Step Installation

- Go to <https://git-scm.com/download/win>
- The download starts automatically
- Run the installer (.exe file)
- During install, accept defaults for almost everything
- Important: When asked about line endings, choose: **Checkout Windows-style, commit Unix-style**
- Important: Choose **Use Git from the Windows Command Prompt** (adds Git to PATH)
- Important: Choose **Use the OpenSSL library** for HTTPS

Verify Installation

Verify

```
git --version
```

Expected output:

```
git version 2.45.x.windows.1
```

TIP » Git Bash is installed alongside Git. It's a Linux-style terminal for Windows. If you prefer Linux commands, you can use it instead of PowerShell.

9. Configuring Git (One-Time Setup)

Why this matters » Git needs to know WHO is making commits. Every commit gets stamped with your name and email. If you don't configure these, your commits will have a generic identity that won't link to your GitHub profile.

The Commands I Ran

Initial Git configuration

```
# Set your name (use your real name)
git config --global user.name "VIJAYKUMAR G CHATTARAKI"

# Set your email (MUST match your GitHub email!)
git config --global user.email "your-github-email@example.com"

# Set default branch name to 'main' (modern standard)
git config --global init.defaultBranch main

# Help Windows remember your GitHub credentials
git config --global credential.helper manager

# Handle Windows line endings
git config --global core.autocrlf true
```

Each Flag Explained

Flag	Meaning
--global	Apply this setting to ALL repos on this machine
user.name	Your display name on commits
user.email	Your email (links commits to GitHub profile)
init.defaultBranch	Default name for new branches when running git init
credential.helper manager	Use Windows Credential Manager to save tokens
core.autocrlf true	Auto-convert line endings (LF to CRLF on checkout)

Verify Configuration

Verify

```
git config --global --list

# Expected output:
user.name=VIJAYKUMAR G CHATTARAKI
user.email=your-github-email@example.com
init.defaultbranch=main
credential.helper=manager
core.autocrlf=true
```

WARNING » Email MUST match the email you used to sign up on GitHub! Otherwise, GitHub won't link your commits to your profile (no green squares on your contribution graph!). Check your GitHub email at: <https://github.com/settings/emails>

MY MISTAKE » My typo: I accidentally configured **credential-manage** (missing 'r' at end) instead of **credential-manager**. The push still worked because Git found the manager anyway, but a warning showed up: **'credential-manage' is not a git command**. To fix: re-run with correct name.

10. Creating .gitignore (Most Critical Step!)

Why this matters » Without .gitignore, you risk committing: passwords, API keys, IDE config, compiled junk, logs, and 100MB+ of build artifacts. ALL of which become PERMANENT in your Git history. Recruiters and bots scan public repos for secrets - this is how AWS keys get stolen!

What .gitignore Does

How .gitignore works

.gitignore is a special file. Git reads it and IGNORES anything matching its patterns.

Example .gitignore:

```
target/      <- ignore everything in target folder
*.log        <- ignore any .log files
.env         <- ignore .env files
!.env.example <- but DO track .env.example (the ! is "not ignored")
```

Effect:

```
git add . will NOT add target/, *.log files, or .env
git status will NOT show them as untracked
git push will NOT send them to GitHub
```

The .gitignore I Used for MediBook

.gitignore for Spring Boot project

```
# =====
# Compiled output & build artifacts
# =====
target/
build/
out/
*.class
*.jar
*.war

# =====
# IntelliJ IDEA
# =====
.idea/
*.iml
*.iws
*.ipr
.idea_modules/

# =====
# Eclipse / STS
# =====
.classpath
.project
.settings/

# =====
# VS Code
# =====
.vscode/

# =====
# OS files
# =====
.DS_Store
Thumbs.db
desktop.ini

# =====
# Logs
# =====
*.log
logs/
spring.log

# =====
# Environment & Secrets - CRITICAL!
# =====
.env
.env.local
.env.*.local
application-secret.yml
application-local.yml
*.pem
*.key

# =====
# Temporary files
# =====
*.tmp
*.bak
*.swp
```


Gitignore Pattern Syntax

Pattern	Matches
target/	Folder named 'target' anywhere
*.log	Any file ending in .log
*.class	Any compiled Java file
.env	Exact filename '.env'
secret*	Any file starting with 'secret'
docs/*.pdf	Only .pdf files directly inside docs/
docs/**/*.pdf	All .pdf files anywhere under docs/
!keep.log	Don't ignore 'keep.log' even if pattern matches
#comment	Comment - ignored by Git

DANGER » NEVER commit: Passwords, API keys, database URLs with credentials, JWT secrets, AWS access keys, SSH private keys, or anything personal. Once committed to a public repo, consider it COMPROMISED forever. Even deleting later doesn't help - Git history preserves it. Rotate any secret that accidentally gets committed!

TIP » Create .gitignore BEFORE the first git add. If you add files first then create .gitignore, those files are already tracked and won't be ignored. Use **git rm --cached file** to untrack.

PART IV · MY ACTUAL JOURNEY (REAL COMMANDS)

This part documents EXACTLY what I ran during Phase 1 Git setup, including my mistakes. Real journey, real errors, real fixes.

11. Command 1: First git init Attempt

What I Ran

First attempt

```
PS C:\projects\medibook-service> git init
```

What Happened

Git initialized a repository. But there was a problem hidden in plain sight - my project's **src/** folder ALREADY had a **.git** folder inside it from before (probably from copying files from another project).

What I Should Have Done First

Pre-flight checks

BEFORE git init, always check:

1. Am I in the right folder?
 pwd (PowerShell) or cd (CMD)
2. Is there ALREADY a .git folder here?
 ls -Force .git (PowerShell)
 dir /a .git (CMD)
3. Are there nested .git folders in subfolders?
 Get-ChildItem -Recurse -Force -Filter ".git" (PowerShell)
 dir /s /b /a:hd .git (CMD)

If yes to #2 or #3: clean up first!

12. Command 2: Discovering the Embedded Git Issue

What I Ran

Stage all files

```
git add .
```

The Scary Output

The warning that confused me

```
warning: in the working copy of '.gitattributes', LF will be replaced by CRLF...
warning: in the working copy of '.gitignore', LF will be replaced by CRLF...
warning: in the working copy of 'pom.xml', LF will be replaced by CRLF...
warning: adding embedded git repository: src
hint: You've added another git repository inside your current repository.
hint: Clones of the outer repository will not contain the contents of
hint: the embedded repository and will not know how to obtain it.
hint: If you meant to add a submodule, use:
hint:   git submodule add <url> src
```

Decoding the Warnings

Warning Type 1: LF / CRLF

These are **harmless**. Different operating systems use different line endings: Linux/Mac use LF (\n), Windows uses CRLF (\r\n). Git is just telling you it will convert line endings on checkout. You can ignore these or silence them with **git config --global core.autocrlf true**.

Warning Type 2: "embedded git repository" (THE REAL PROBLEM)

DANGER » This is critical! It means Git found a **.git** folder INSIDE your project. Git treats it as a separate repository (submodule). If you push this to GitHub, the **src/** folder will be EMPTY on GitHub because Git only stores a reference, not the actual files!

Why This Happened to Me

Investigation

Possible causes:

1. I copied src/ from another Git project (most likely)
2. I accidentally ran git init INSIDE src/
3. Spring Initializr download had something weird
4. Old code I imported had .git folder inside

How to verify:

```
Get-ChildItem -Recurse -Force -Directory -Filter ".git"
```

If you see TWO results = embedded git problem!

MY MISTAKE » I almost pushed broken code to GitHub. If I hadn't paused to verify, my src/ folder would have been empty on GitHub - completely useless for anyone trying to use my code.

13. Command 3: Locating the Rogue .git Folder

What I Ran

Find all .git folders

```
Get-ChildItem -Path . -Recurse -Force -Directory -ErrorAction SilentlyContinue |  
  Where-Object { $_.Name -eq ".git" } |  
  Select-Object FullName
```

Each Part Explained

Part	Meaning
Get-ChildItem	PowerShell cmdlet to list folder contents (like 'ls' or 'dir')
-Path .	Start searching from current directory (.)
-Recurse	Search inside ALL subfolders (recursively)
-Force	Include hidden files/folders (.git is hidden!)
-Directory	Only list folders, not files
-ErrorAction SilentlyContinue	Don't crash on permission errors, just skip
	Pipe - send output of one command into the next
Where-Object	Filter the list with a condition
{ \$_.Name -eq ".git" }	Keep only items where Name equals '.git'
Select-Object FullName	Only show the FullName property (not all 20+ properties)

My Output

Two .git folders found

FullName

```
C:\projects\medibook-service\.git          <- The main repo .git (correct)  
C:\projects\medibook-service\src\.git      <- ROGUE! This is the problem!
```

WHAT I LEARNED » Hidden files are a common source of bugs. Always use **-Force** in PowerShell or **/a** in CMD when investigating. The .git folder is hidden by default on Windows.

14. Command 4: Killing the Inner .git

What I Ran (After Failed CMD Attempt)

MY MISTAKE » FIRST I TRIED THIS (WRONG IN POWERSHELL):

```
rmdir /s /q .git
```

Error received:

"A positional parameter cannot be found that accepts argument '/q'."

Why: PowerShell's **rmdir** is actually **Remove-Item** in disguise (an alias). Remove-Item uses different syntax. The /s and /q flags are CMD-only.

The Correct PowerShell Command

Delete the inner .git folder

```
Remove-Item -Path "src\.git" -Recurse -Force
```

Every Part Explained

Part	Meaning
Remove-Item	PowerShell cmdlet to delete files or folders
-Path "src\.git"	What to delete (relative path)
-Recurse	Delete folder AND all its contents (without this, fails for non-empty folders)
-Force	Force deletion even for hidden/read-only files

CMD Equivalent

If you were in CMD instead of PowerShell

```
rmdir /s /q src\.git
```

Part	Meaning
rmdir	CMD command to remove directory
/s	Recursively delete subdirectories
/q	Quiet mode - don't ask for confirmation
src\.git	Path to folder to delete

WARNING » BE CAREFUL! -Force and -Recurse together = DANGEROUS. Triple-check the path before running. Once deleted, files don't go to the Recycle Bin - they're gone forever (well, until your next backup runs).

15. Command 5: Verification

Why Verify?

After deleting something, ALWAYS verify it's actually gone. Sometimes deletes fail silently (permission issues, files in use, etc.). I didn't trust my eyes - I ran the search command again.

What I Ran

Same search as before

```
Get-ChildItem -Path . -Recurse -Force -Directory -ErrorAction SilentlyContinue |  
    Where-Object { $_.Name -eq ".git" } |  
    Select-Object FullName
```

Output (After Successful Delete)

Only one .git folder - perfect!

```
FullName  
-----  
C:\projects\medibook-service\.git
```

WHAT I LEARNED » Always verify after destructive operations. 'Trust but verify' is a core principle in production engineering. The 10 seconds spent verifying saves hours of debugging later.

16. Command 6: Resetting the Stage

Why This Was Needed

The previous **git add .** had staged things INCORRECTLY (with `src` as a submodule reference). Even though I deleted the rogue `.git`, the staging area still had the bad reference. I had to **unstage** everything and start fresh.

What I Ran

Unstage everything

```
git rm -rf --cached .
```

Every Part Explained

Part	Meaning
git rm	Git command to remove files
-r	Recursive (apply to subfolders)
-f	Force (don't fail on errors)
--cached	Only remove from staging area, KEEP file on disk
.	All files (current directory and subdirectories)

DANGER » Without `--cached`, this command would DELETE YOUR FILES from disk! **git rm -rf .** would wipe your entire project. The **`--cached`** flag is the difference between safety and disaster. ALWAYS use `--cached` when you want to unstage but keep files.

Mental Model

rm vs rm --cached

```
git rm file.txt           = Delete from staging AND disk
git rm --cached file.txt  = Delete from staging, KEEP on disk
```

```
git rm -r folder/        = Delete folder + contents (staging + disk)
git rm -r --cached folder/= Untrack folder (keep on disk)
```

The `--cached` flag is the SAFETY catch.

My Output

Files unstaged (still on disk!)

```
rm '.gitattributes'
rm '.gitignore'
rm '.mvn/wrapper/maven-wrapper.properties'
rm 'mvnw'
rm 'mvnw.cmd'
rm 'pom.xml'
rm 'src'
rm 'qodana.yaml'
```

<- Notice: just 'src' (the bad reference)

17. Command 7: Fresh git add

What I Ran

Stage everything (clean attempt)

```
git add .
```

Output This Time (THE GOOD ONE)

Only line-ending warnings - no embedded git!

```
warning: in the working copy of '.gitattributes', LF will be replaced by CRLF...
warning: in the working copy of '.gitignore', LF will be replaced by CRLF...
warning: in the working copy of 'pom.xml', LF will be replaced by CRLF...
warning: in the working copy of 'src/main/java/.../MedibookServiceApplication.java', LF will be replaced...
warning: in the working copy of 'src/test/java/.../MedibookServiceApplicationTests.java', LF will be replac
```

TIP » Critical difference from before: No more "adding embedded git repository: src" warning! Plus, individual files inside **src/** are now being staged. This means the fix worked.

18. Command 8: Verifying Staging

What I Ran

Check what's staged

```
git status
```

Output (Beautiful!)

34 files staged correctly

On branch main

No commits yet

Changes to be committed:

(use "git rm --cached <file>..." to unstage)

```
new file:   .gitattributes
new file:   .gitignore
new file:   .mvn/wrapper/maven-wrapper.properties
new file:   mvnw
new file:   mvnw.cmd
new file:   pom.xml
new file:   qodana.yaml
new file:   src/main/java/com/medibook/medibookservice/MedibookServiceApplication.java
new file:   src/main/java/com/medibook/medibookservice/config/SecurityConfig.java
new file:   src/main/java/com/medibook/medibookservice/controller/AuthController.java
new file:   src/main/java/com/medibook/medibookservice/controller/UserController.java
... (34 files total)
```

What to Look For

- **Branch:** 'On branch main' (good - using modern default)
- **Status:** 'No commits yet' (expected - first commit)
- **Individual files in src/main/java/...** (no more 'src' as single entry!)
- **All YAML files:** application.yml, application-dev.yml, application-prod.yml
- **NO .idea/ folder files** (correctly ignored)
- **NO target/ folder files** (correctly ignored)
- **NO .env files** (correctly ignored)

Why this matters » `git status` is your best friend. Use it constantly. Before every commit. After every operation. It tells you exactly what state your repo is in. If something feels wrong, run `git status` first.

19. Command 9: First Commit

What I Ran

First commit with detailed message

```
git commit -m "feat: complete Phase 1 - authentication module" \  
-m "- Add User entity with JPA auditing and indexed email" \  
-m "- Add UserRepository with findByEmail and existsByEmail" \  
-m "- Add Register and Login DTOs with Bean Validation" \  
-m "- Add custom exceptions and global exception handler" \  
-m "- Add UserMapper for Entity-DTO conversion" \  
-m "- Implement JWT authentication using JJWT 0.12.6"
```

Every Part Explained

Part	Meaning
git commit	Create a snapshot from staged files
-m	Provide message inline (avoid opening text editor)
"..."	The commit message (in quotes)
Multiple -m	Each becomes a paragraph in the final message

Why Multiple -m Flags?

PowerShell handles multi-line strings differently from CMD. Using multiple **-m** flags is the safest way to get a multi-paragraph commit message that works in both shells.

My Output

First commit created successfully

```
[main (root-commit) c5cd306] feat: complete Phase 1 - authentication module  
34 files changed, 2104 insertions(+)  
create mode 100644 .gitattributes  
create mode 100644 .gitignore  
create mode 100644 pom.xml  
create mode 100644 src/main/java/com/medibook/medibookservice/MedibookServiceApplication.java  
create mode 100644 src/main/java/com/medibook/medibookservice/config/SecurityConfig.java  
... (more file creation lines)
```

Decoding the Output

Part	Meaning
[main	Branch where commit was made
(root-commit)	This is the FIRST commit ever (no parent)
c5cd306]	Commit hash (short version) - unique ID
feat: complete Phase 1...	Your commit message
34 files changed	Number of files affected

Part	Meaning
2104 insertions(+)	Lines added (0 deletions for first commit)
create mode 100644	File permissions (100644 = regular non-executable file)

TIP » Every commit has a unique hash (SHA-1). You can reference any commit by its hash. Even just the first 7 characters are usually enough: **git show c5cd306**

20. Command 10: Connecting to GitHub

Prerequisite: Create GitHub Repo First

- Go to <https://github.com>
- Click + icon » New repository
- Repository name: **MediBook**
- Description: "Doctor Appointment Booking System"
- Visibility: **Public** (good for portfolio)
- **DO NOT** initialize with README, .gitignore, or license (we have these already)
- Click **Create repository**

What I Ran

Connect local repo to GitHub

```
git remote add origin https://github.com/VIJAYKUMAR-G-CHATTARAKI/MediBook.git
```

Every Part Explained

Part	Meaning
git remote	Manage connections to remote repositories
add	Add a new remote (vs remove, rename, set-url)
origin	Conventional name for the primary remote (you could call it anything)
https://github.com/...	URL of your GitHub repository

Why 'origin'?

origin is just a name - a convention. Like calling your dog "Rex" instead of "My Pet #1". You could name it anything (**github**, **main-repo**, **upstream**), but using **origin** is the universal standard everyone recognizes.

Verify Remote

Verify connection

```
git remote -v
```

Expected output (-v means verbose):

```
origin https://github.com/VIJAYKUMAR-G-CHATTARAKI/MediBook.git (fetch)
origin https://github.com/VIJAYKUMAR-G-CHATTARAKI/MediBook.git (push)
```

Two lines appear because Git tracks fetch (download) and push (upload) URLs separately. Usually they're the same, but advanced setups can have different URLs for each.

MY MISTAKE » Common mistake: Adding 'origin' twice. If you accidentally run **git remote add origin URL** twice, you get: **"error: remote origin already exists."** Fix: **git remote set-url origin NEW_URL** to update, or **git remote remove origin** to delete and start over.

21. Command 11: The Push

What I Ran

Upload commits to GitHub

```
git push -u origin main
```

Every Part Explained

Part	Meaning
git push	Upload commits to remote repository
-u	Set upstream tracking (so future 'git push' alone works)
origin	Which remote to push to
main	Which local branch to push

What Does -u Do Exactly?

Why -u is convenient

Without -u:

```
git push origin main    <- have to specify every time
```

With -u (first push only):

```
git push -u origin main  <- creates a link: local main <-> origin/main
```

After -u, you can just use:

```
git push    <- Git knows where to push (uses tracking)
git pull    <- Git knows where to pull from
```

Authentication

When you push for the first time, Git will prompt for credentials:

Credential prompt

Username: VIJAYKUMAR-G-CHATTARAKI

Password: [PERSONAL ACCESS TOKEN, NOT YOUR GITHUB PASSWORD]

DANGER » GitHub removed password authentication in 2021! You MUST use a Personal Access Token (PAT). If you paste your GitHub password, you'll get 'Authentication failed'.

My Successful Output

Successful push

```
Enumerating objects: 63, done.
Counting objects: 100% (63/63), done.
Delta compression using up to 12 threads
Compressing objects: 100% (48/48), done.
Writing objects: 100% (63/63), 26.19 KiB | 596.00 KiB/s, done.
Total 63 (delta 4), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (4/4), done.
To https://github.com/VIJAYKUMAR-G-CHATTARAKI/MediBook.git
 * [new branch]      main -> main
branch 'main' set up to track 'origin/main'
```

Decoding Push Output

Line	Meaning
Enumerating objects	Git is preparing what to send
Counting objects	Counting files to upload
Compressing objects	Git compresses to save bandwidth
Writing objects	Actually uploading (with size + speed)
[new branch] main -> main	Created new branch on remote
set up to track	-u flag worked - now linked to remote

22. Command 12: Creating Feature Branch

What I Ran

Create and switch to new branch

```
git checkout -b feature/phase-2-doctor-module
```

Every Part Explained

Part	Meaning
git checkout	Switch to a branch (legacy command - still works)
-b	Create the branch if it doesn't exist
feature/phase-2-doctor-module	Branch name (slashes are allowed)

Modern Alternative

Newer command - same effect

Modern Git (2.23+) prefers:

```
git switch -c feature/phase-2-doctor-module
```

Both work. **checkout** is older, more universal. **switch** is newer, more specific. Most tutorials still use **checkout**, so it's safer to learn first.

Branch Naming Conventions

Prefix	Used For
feature/	New features (most common)
fix/	Bug fixes
hotfix/	Urgent production fixes
release/	Release preparation
chore/	Maintenance tasks (dependency updates, etc.)
docs/	Documentation only changes
refactor/	Code restructuring
test/	Adding or fixing tests

TIP » Use kebab-case for branch names (words separated by hyphens): **feature/user-authentication**, not **feature/UserAuthentication**. Avoids case sensitivity issues across Windows/Mac/Linux.

PART V · DEEP DIVE - EVERY FLAG EXPLAINED

23. git init - Initialize a Repository

Purpose

Creates a new **.git** folder in your current directory, turning a regular folder into a Git repository.

Common Usage

git init variants

```
git init                # Initialize in current folder
git init my-project     # Create new folder + initialize
git init --bare my-repo.git # Bare repo (no working files, for servers)
```

Common Flags

Flag	Purpose
--bare	Create bare repo (used on servers, no working directory)
--initial-branch=NAME	Set initial branch name (default is 'main' if configured)
-b NAME	Short for --initial-branch
--template=DIR	Use template directory for new repo

What It Actually Creates

Inside **.git** folder

```
my-project/
+-- .git/
    +-- HEAD          <- Points to current branch
    +-- config         <- Repo-specific config
    +-- description    <- Repo description
    +-- hooks/         <- Custom scripts (pre-commit, etc.)
    +-- info/          <- Exclude file (like .gitignore but local-only)
    +-- objects/       <- Stores all your commits/files (compressed)
    +-- refs/          <- Branch and tag references
        +-- heads/     <- Local branches
        +-- tags/      <- Tags
```

WARNING » Never edit files inside **.git/** manually unless you know what you're doing. You can corrupt your repo permanently.

24. git add - Stage Files

Purpose

Moves changes from working directory to staging area (preparing for commit).

Common Usage

git add variants

```
git add .           # Stage all changes in current folder
git add filename.java # Stage one specific file
git add src/        # Stage all changes in src folder
git add *.java      # Stage all .java files
git add -A          # Stage everything (including deletions, anywhere)
git add -u          # Stage modifications + deletions only (NO new files)
git add -p          # Interactive: review each change before staging
```

The Difference Between . and -A

. vs -A vs -u

```
git add .           Stages everything in CURRENT directory and below
                    (newer Git versions also include deletions)
```

```
git add -A          Stages everything in the ENTIRE working tree
                    (no matter what subfolder you're in)
```

```
git add -u          Stages only MODIFIED and DELETED files
                    (skips new untracked files)
```

In practice: `git add .` is what you'll use 95% of the time.

Common Flags

Flag	Purpose
.	All files in current dir and below
-A	All files in entire repo
-u	Only update tracked files (no new files)
-p	Patch mode - review each chunk interactively
-i	Interactive mode (menu-driven)
--dry-run	Show what WOULD be added without doing it
-v	Verbose - show file names as they're added
-f	Force - add even ignored files

TIP » Use `git add -p` to review changes chunk by chunk before staging. Great for splitting one set of changes into multiple commits.

25. git status - Check State

Purpose

Shows which files are modified, staged, untracked, or ignored. Your most-used command!

Common Usage

git status variants

```
git status          # Full status (default)
git status -s       # Short version (one line per file)
git status --ignored # Also show ignored files
git status -uno      # Hide untracked files
```

Short Status Codes

Short codes

```
git status -s

# Output uses 2-character codes:
M file.txt      <- Modified, not staged
M file.txt      <- Modified, staged
MM file.txt     <- Modified, staged, then modified again
A file.txt      <- New file, staged
D file.txt      <- Deleted, staged
?? file.txt     <- Untracked (new file, not staged)
!! file.txt     <- Ignored
```

TIP » After getting comfortable, **git status -s** is faster to read than full output.

26. git commit - Save a Snapshot

Purpose

Creates a permanent snapshot of staged changes with a message describing what changed.

Common Usage

git commit variants

```
git commit -m "Your message here"
```

```
# Multi-paragraph message (PowerShell-friendly):
```

```
git commit -m "Title line" -m "First paragraph" -m "Second paragraph"
```

```
# Open editor for longer message:
```

```
git commit
```

```
# (Will open vim/nano - use git config to set your preferred editor)
```

```
# Amend last commit (forgot a file, fix typo in message):
```

```
git commit --amend -m "New message"
```

```
# Add staged changes to last commit (DON'T use after push!):
```

```
git commit --amend --no-edit
```

Common Flags

Flag	Purpose
-m "msg"	Commit message inline
-a	Auto-stage modified files (skips new files - dangerous)
-am "msg"	Combine -a and -m
--amend	Replace the last commit
--no-edit	Don't change message when amending
--allow-empty	Create commit with no changes (useful for triggering CI)
--author="Name <email>"	Override author for this commit
-S	Sign commit with GPG key (verified badge on GitHub)

DANGER » NEVER amend a commit you've already pushed to a shared branch! Amending changes the commit hash. Teammates who pulled the old commit will have conflicts. Only amend LOCAL commits that haven't been pushed yet.

27. git remote - Connect to GitHub

Common Operations

git remote operations

<code>git remote -v</code>	<code># List all remotes (verbose)</code>
<code>git remote add origin URL</code>	<code># Add a new remote</code>
<code>git remote remove origin</code>	<code># Remove a remote</code>
<code>git remote rename origin upstream</code>	<code># Rename a remote</code>
<code>git remote set-url origin NEW_URL</code>	<code># Change a remote's URL</code>
<code>git remote show origin</code>	<code># Show details about a remote</code>

HTTPS vs SSH URLs

Two URL formats

`# HTTPS (uses PAT for auth)`
`https://github.com/username/repo.git`

`# SSH (uses SSH key for auth)`
`git@github.com:username/repo.git`

HTTPS is easier for beginners - just need a PAT. **SSH** requires setting up SSH keys but is more secure and convenient long-term (no password prompts). For Phase 1, we used HTTPS.

28. git push - Upload to GitHub

Common Usage

git push variants

```
git push                # Push current branch to its tracking remote
git push origin main    # Push 'main' branch to 'origin' remote
git push -u origin main  # Set upstream (first push)
git push --all          # Push all branches
git push --tags         # Push tags (versions)
git push --force        # Overwrite remote (DANGEROUS!)
git push --force-with-lease # Safer force (checks if anyone else pushed)
git push origin --delete branch-name # Delete remote branch
```

DANGER » NEVER use git push --force on shared branches! It overwrites teammates' work. If you must force, use **--force-with-lease** which aborts if anyone else has pushed. Force-push only on YOUR feature branches that nobody else uses.

29. git rm - Remove Files

Three Variations

git rm variants

```
git rm file.txt           # Delete from staging AND disk
git rm --cached file.txt  # Untrack but keep on disk
git rm -r folder/         # Delete folder + contents (staging + disk)
git rm -r --cached folder/ # Untrack folder (keep files)
git rm -rf --cached .     # Untrack EVERYTHING (force, recursive)
```

Flags Explained

Flag	Purpose
-r	Recursive (required for folders)
-f	Force (skip checks, don't ask)
--cached	Only unstage, keep file on disk
--dry-run	Preview what would be removed
-q	Quiet - don't show files

WARNING » Mnemonic: **--cached = safety lock**. Without it, files DIE. With it, files SURVIVE on disk. When in doubt, add --cached. You can always delete files manually later.

30. git checkout - Switch Branches

Multiple Uses (Confusing!)

git checkout uses

```
git checkout main                # Switch to main branch
git checkout -b new-branch       # Create + switch to new branch
git checkout filename.txt       # Discard changes to a file (revert!)
git checkout -- .                # Discard ALL local changes (DANGEROUS)
git checkout abc123              # Switch to a specific commit (detached HEAD)
git checkout main -- file.txt    # Get file from main branch into current
```

Modern Alternative (Recommended)

Modern Git (2.23+)

```
# git switch - for branches (clearer):
git switch main                # Switch to existing branch
git switch -c new-branch       # Create + switch (-c instead of -b)

# git restore - for files (clearer):
git restore file.txt           # Discard changes to a file
git restore --staged file.txt  # Unstage a file
```

TIP » New commands **git switch** and **git restore** split the confusing **git checkout** into specific tools. Use them when you can.

31. git branch - Manage Branches

git branch operations

<code>git branch</code>	<code># List local branches (* = current)</code>
<code>git branch -a</code>	<code># List local + remote branches</code>
<code>git branch -r</code>	<code># List only remote branches</code>
<code>git branch new-feature</code>	<code># Create branch (don't switch)</code>
<code>git branch -d old-feature</code>	<code># Delete branch (safe - won't delete unmerged)</code>
<code>git branch -D old-feature</code>	<code># FORCE delete (loses unmerged work!)</code>
<code>git branch -m old-name new-name</code>	<code># Rename branch</code>
<code>git branch --merged</code>	<code># List branches merged into current</code>
<code>git branch --no-merged</code>	<code># List branches NOT merged</code>

32. git log - View History

git log variants

```
git log                # Full history (most recent first)
git log --oneline       # One line per commit (compact)
git log --graph         # ASCII art branch visualization
git log --oneline --graph --all # Pretty visualization of all branches
git log -n 5            # Only last 5 commits
git log --author="Vijay" # Commits by specific author
git log --since="2 weeks ago" # Commits in last 2 weeks
git log --until="yesterday"  # Commits before yesterday
git log -- filename      # History of a specific file
git log -p               # Show full diff for each commit
git log --stat           # Show file change statistics
```

Pretty Format

Pretty git log alias

Save as alias - looks beautiful!

```
git config --global alias.lg "log --color --graph --pretty=format:'%Cred%h%Creset -%C(yellow)%d%Creset %s %'
```

Now use:

```
git lg
```

PART VI · GITHUB SETUP

33. Creating a GitHub Account

- Go to <https://github.com/signup>
- Choose a professional username (recruiters see this!)
- Use a real email (you'll use it in Git config)
- Verify your email
- Free plan is perfect for unlimited public repos

TIP » Pick a username that you'd put on your resume. **VIJAYKUMAR-G-CHATTARAKI** is much better than **xX_CodeMaster123_Xx**.

34. Creating Your First Repository

- Click + (top right) » New repository
- Repository name: **MediBook**
- Description: "Doctor Appointment Booking System"
- Visibility: Public (great for portfolio)
- **DO NOT** add README, .gitignore, or LICENSE (we have them locally)
- Click Create repository
- Copy the URL: **<https://github.com/USERNAME/MediBook.git>**

WARNING » If you accidentally initialize with README on GitHub, you'll get push conflicts. Either delete that file on GitHub first, or run **git pull --rebase origin main** before pushing.

35. Personal Access Token (PAT) Setup

Why this matters » GitHub disabled password authentication in August 2021. You **MUST** use a Personal Access Token (PAT) instead of your password for HTTPS push/pull. This is more secure - tokens can be revoked individually, have expiration dates, and limited permissions.

Step-by-Step PAT Creation

- Visit <https://github.com/settings/tokens>
- Click **Generate new token (classic)**
- Settings:
 - **Note:** medibook-laptop (description for your reference)
 - **Expiration:** 90 days (or whatever you prefer)
 - **Scopes:** Check **repo** (full control of private repos)
 - Click **Generate token** at bottom
 - **COPY THE TOKEN IMMEDIATELY** (starts with ghp_...)
 - Save it in a password manager - YOU WILL NEVER SEE IT AGAIN

DANGER » Treat your PAT like a password! Anyone with this token has full access to your GitHub repos. If you accidentally expose it (commit it, share it), **REVOKE IT IMMEDIATELY** at <https://github.com/settings/tokens>

Using PAT in Git Push

Authentication

```
git push -u origin main
```

Git prompts:

Username: VIJAYKUMAR-G-CHATTARAKI

Password: ghp_XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX

|

+-- PASTE YOUR PAT HERE (not your password!)

TIP » Set **git config --global credential.helper manager** to make Windows save your PAT. After one successful push, you won't need to enter it again on this machine.

36. Why Password Auth Was Removed

Until August 2021, you could just type your GitHub password to push. Then GitHub removed it. Why?

Security Reasons

- **Passwords are reusable across sites** - if leaked once, breaks everything
- **No granular permissions** - password = full account access
- **No expiration** - leaked password works forever until you change it
- **No revocation tracking** - can't tell which device leaked
- **No 2FA enforcement** - password bypasses 2FA on Git operations

Benefits of PATs

- **Granular scopes** - only give what's needed (e.g., only 'repo' access)
- **Expiration dates** - automatically invalid after period
- **Easy revocation** - delete one token without affecting others
- **Per-device tokens** - one for laptop, one for desktop, etc.
- **Audit trail** - see which token was used for what

TIP » Future-proof: GitHub recommends moving to **SSH keys** or **GitHub CLI (gh)** for long-term use. PATs are fine for now but require renewal.

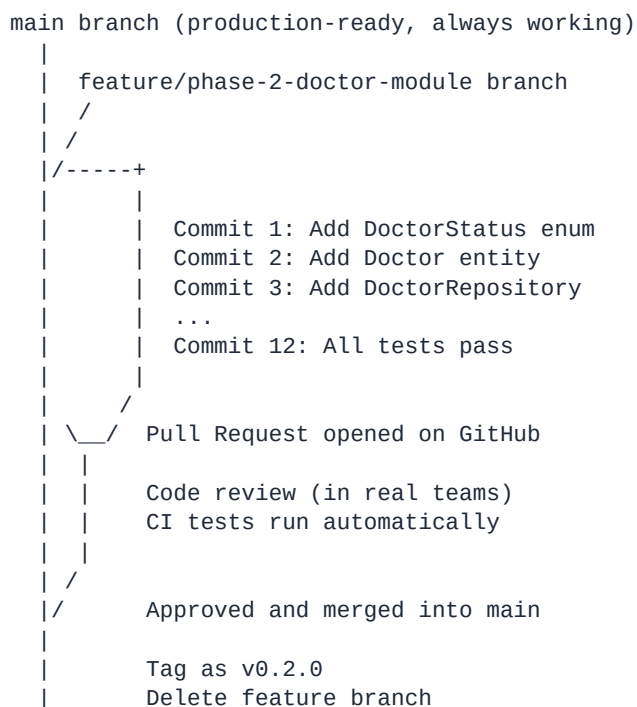
PART VII · PROFESSIONAL WORKFLOW

37. Feature Branch Strategy

Why this matters » Working directly on **main** is amateur. Professional teams ALWAYS use feature branches. Why? **main** should always be deployable. Untested code shouldn't live there. Feature branches isolate experiments until they're proven good.

The Workflow

Feature branch lifecycle



Branch Naming Conventions

Type	Format	Example
Feature	feature/short-description	feature/doctor-module
Bug fix	fix/short-description	fix/login-validation
Hotfix	hotfix/issue-description	hotfix/production-500-error
Release	release/version	release/v1.2.0
Chore	chore/task	chore/upgrade-spring-boot
Docs	docs/topic	docs/api-readme

TIP » Branch names should be DESCRIPTIVE but SHORT. **feature/add-doctor-approval-workflow-with-admin-controls** is too long. **feature/doctor-approval** says enough.

38. Conventional Commit Messages

Why this matters » Random commit messages like "updated stuff" are useless. Conventional Commits is a specification used by Google, Angular, Vue, and most major projects. It enables: automatic changelogs, semantic versioning, easier code review.

The Format

Format

`<type>(<scope>): <subject>`

`<body>`

`<footer>`

Types

Type	When to Use	Example
feat	New feature for the user	feat(auth): add JWT login
fix	Bug fix for the user	fix(auth): handle expired tokens
docs	Documentation only	docs(readme): add setup steps
style	Formatting, no code change	style: format with prettier
refactor	Code restructure (no behavior change)	refactor: extract validator class
perf	Performance improvement	perf(db): add index on email
test	Adding or fixing tests	test(auth): add login error tests
chore	Maintenance tasks	chore: upgrade dependencies
ci	CI/CD config changes	ci: add GitHub Actions workflow
revert	Revert a previous commit	revert: undo broken commit abc123

Real Examples (Good)

Good commit messages

`feat(doctor): add DoctorStatus enum`

`fix(security): prevent email enumeration in login error`

`docs(readme): document Phase 2 endpoints`

`refactor(service): extract password validation to utility`

`test(auth): add integration tests for register endpoint`

`chore(deps): bump Spring Boot to 3.5.14`

Bad Examples

Bad commit messages

fixed stuff	<- What stuff?
update	<- Update what?
asdf	<- Useless
wip	<- Work in progress - shouldn't be committed
final	<- Nothing is ever final
fixed the bug again	<- Which bug? Why "again"?

TIP » Imagine reading your commits 6 months from now. Would past-you understand what you did? Write for that future reader.

39. Pull Request Workflow

A **Pull Request (PR)** is a request to merge your feature branch into **main**. It's where code review happens. This is the heartbeat of professional development.

The PR Lifecycle

PR workflow

1. CREATE PR
 - Push feature branch to GitHub
 - Click "New Pull Request" on GitHub
 - Add title (use conventional commits format)
 - Add description (what changed, why, how to test)
 - Link related issues (#42 fixes #38)
 - Add reviewers
2. AUTOMATED CHECKS RUN
 - CI/CD pipeline triggered automatically
 - Tests run
 - Linters check code style
 - Security scans run
3. CODE REVIEW
 - Reviewer reads code line by line
 - Comments on issues, suggestions
 - May request changes
4. ADDRESS FEEDBACK
 - Make changes in your branch
 - Push again - PR updates automatically
 - Reply to reviewer comments
5. APPROVED
 - Reviewer approves
 - CI passes
6. MERGE
 - Three options:
 - a) Merge commit (preserves all commit history)
 - b) Squash and merge (combines into one commit)
 - c) Rebase and merge (linear history)
7. CLEAN UP
 - Delete the feature branch
 - Pull latest main locally

Good PR Description Template

PR description

What

Add Doctor approval workflow with admin controls.

Why

Doctors need verification before appearing publicly to ensure only legitimate medical professionals can receive bookings.

How

- Added DoctorStatus enum (PENDING_APPROVAL, APPROVED, REJECTED, SUSPENDED)
- Added approval fields to Doctor entity
- Created /admin/doctors/pending endpoint
- Created /admin/doctors/{id}/approve endpoint
- Created /admin/doctors/{id}/reject endpoint

Testing

- All 11 Postman tests pass
- Tested with PATIENT, DOCTOR, ADMIN roles
- Verified rejected doctors can resubmit when allowed

Screenshots

[Add Postman screenshots]

Related Issues

Closes #15

40. Commit Rhythm - When to Commit

Rule of Thumb

Commit when you reach a **logical stopping point** - something that works on its own, even if incomplete. NOT after every line.

Good Commit Boundaries

- After adding a new file that compiles
- After implementing one method that works
- After fixing one bug
- After adding one feature (small ones)
- Before starting something risky (safety net)
- Before lunch / end of day (don't lose work)

Bad Commit Examples

Commit sizing

TOO BIG (one commit = too much):

Commit: "Added entire user module"

- 15 files
- 1500 lines
- Auth + DTOs + Service + Repository + Tests + Config

Problem: code review nightmare, can't revert just one piece

TOO SMALL (excessive commits):

Commit 1: "Added blank line"

Commit 2: "Fixed typo in comment"

Commit 3: "Renamed variable"

Commit 4: "Renamed it back"

Problem: cluttered history, useless to bisect

JUST RIGHT:

Commit 1: "feat(auth): add User entity"

Commit 2: "feat(auth): add UserRepository"

Commit 3: "feat(auth): add register endpoint"

Commit 4: "test(auth): add register tests"

Push Rhythm (Different from Commit!)

Commit OFTEN (locally). Push when a complete feature works. Push = sharing with team. Don't push broken code. But also don't hoard commits forever - push at least once a day.

PART VIII · REFERENCE

41. Mistakes I Made and What I Learned

Mistake 1: Used CMD Syntax in PowerShell

MY MISTAKE » Ran `rmdir /s /q .git` in PowerShell. Got: "A positional parameter cannot be found that accepts argument '/q'."

WHAT I LEARNED » PowerShell aliases like `rmdir` point to different commands than CMD. Always check which shell you're in (look for **PS** prefix). When copying tutorials, verify the shell they assume.

Mistake 2: Didn't Check for Embedded Git

MY MISTAKE » Ran `git init` without verifying no nested `.git` folders existed. Got the dreaded "adding embedded git repository: src" warning. Would have pushed empty `src/` to GitHub!

WHAT I LEARNED » Before any `git init`, ALWAYS check for existing `.git` folders recursively. Hidden `.git` folders are silent killers.

Mistake 3: Default Branch Was 'master'

MY MISTAKE » Ran `git init` and got branch named `master` instead of `main`. GitHub now defaults to `main`. Without alignment, push commands fail.

WHAT I LEARNED » Set `git config --global init.defaultBranch main` once. All future repos will use `main` automatically.

Mistake 4: Typo in Credential Helper Config

MY MISTAKE » Configured `credential-manage` (missing 'r') instead of `credential-manager`. Got warning: '`credential-manage`' is not a git command.

WHAT I LEARNED » Always verify config after setting. Use `git config --global --list` to inspect.

Mistake 5: Wanted to Commit Everything at Once

MY MISTAKE » Initial instinct was to make ONE big commit for Phase 1. Bad idea - hard to review, hard to bisect bugs, ugly history.

WHAT I LEARNED » Atomic commits per logical change. Even retroactively breaking work into smaller commits beats one giant commit.

42. Quick Reference Cheat Sheet

Daily Workflow

Daily commands

# Start of day	
git pull	# Get latest from remote
# Make changes... then:	
git status	# See what changed
git diff	# Review changes line by line
git add .	# Stage everything
git commit -m "feat(scope): description"	# Commit
git push	# Upload to remote
# Branch work	
git checkout -b feature/my-feature	# New feature branch
# (make commits)	
git push -u origin feature/my-feature	# First push needs -u
# After PR merged	
git checkout main	# Switch to main
git pull	# Get the merge
git branch -d feature/my-feature	# Delete local branch

Inspecting

Inspection commands

git status	# What's the current state?
git log --oneline	# Quick history
git log --graph --all	# Visual branch history
git diff	# What's changed (unstaged)?
git diff --staged	# What's staged?
git show abc123	# Show specific commit
git blame file.txt	# Who changed each line?

Undoing Things

Undo commands

```
# Discard uncommitted changes to a file
git restore file.txt
# OR (old style):
git checkout -- file.txt

# Unstage a file (keep changes)
git restore --staged file.txt
# OR (old style):
git reset HEAD file.txt

# Undo last commit (keep changes staged)
git reset --soft HEAD~1

# Undo last commit (discard changes - DANGEROUS!)
git reset --hard HEAD~1

# Amend last commit message
git commit --amend -m "New message"

# Add forgotten file to last commit
git add forgotten.txt
git commit --amend --no-edit

# Revert a commit (creates new commit that undoes it)
git revert abc123
```

Stashing (Save Work Temporarily)

Stash commands

git stash	# Save uncommitted changes
git stash list	# See saved stashes
git stash pop	# Apply latest stash and remove it
git stash apply	# Apply but keep in stash
git stash drop	# Delete latest stash
git stash branch new-branch	# Apply stash to new branch

43. Recovery from Common Disasters

"I committed to the wrong branch!"

Wrong branch fix

```
# You committed to main, should have been on feature branch
git log --oneline -1          # Note the commit hash (e.g., abc123)
git reset --hard HEAD~1      # Remove commit from main (KEEP CHANGES)
git checkout feature-branch  # Switch to correct branch
git cherry-pick abc123        # Apply the commit here
```

"I committed sensitive data (passwords)!"

DANGER » REVOKE THE SECRET IMMEDIATELY! Don't waste time removing from history first - bots already scraped it. Change the password / regenerate the API key / revoke the token. THEN clean up history:

Secret removal

```
# Remove from history (rewrites history - careful!)
git filter-branch --force --index-filter \
  "git rm --cached --ignore-unmatch path/to/secret-file" \
  --prune-empty --tag-name-filter cat -- --all

# Force push (only if you're the only one using the branch)
git push --force --all

# Better tool for this:
# Install BFG Repo Cleaner and use it instead
```

"My local repo is broken!"

Re-clone fresh

```
# Nuclear option - re-clone fresh from GitHub
cd ..
mv broken-repo broken-repo-backup
git clone https://github.com/USERNAME/REPO.git
```

"I deleted my branch by accident!"

Branch recovery

```
# Find the deleted branch's last commit
git reflog                    # Lists ALL recent HEAD movements

# Look for "checkout: moving from your-branch"
# Note the commit hash (e.g., abc123)

# Restore the branch
git branch your-branch abc123
```

"Merge conflict! I'm scared!"

Merge conflict resolution

Don't panic. Conflicts are normal.

Files in conflict show with conflict markers:

<<<<<< HEAD

Your version

=====

Their version

>>>>>> branch-name

To fix:

1. Open conflicted file

2. Decide which version to keep (or combine)

3. Remove the <<<<, =====, >>>> markers

4. Save the file

5. Mark as resolved:

git add path/to/file

When all conflicts resolved:

git commit # Completes the merge

If you give up:

git merge --abort # Cancel the merge entirely

44. Interview Talking Points

O: Walk me through your Git workflow

WHAT I LEARNED » "I follow trunk-based development with short-lived feature branches. I create a branch like **feature/doctor-module**, make small atomic commits using conventional commit format (**feat**, **fix**, **docs**), then open a Pull Request for review before merging to main. I never commit secrets - **.gitignore** is the first file I add to any project."

O: How do you write good commit messages?

WHAT I LEARNED » "I use Conventional Commits format: type(scope): subject. For example, **feat(auth): add JWT token refresh**. The type tells you WHAT kind of change (feature, fix, docs), the scope tells you WHERE in the codebase, and the subject is a brief description. This format enables automated changelog generation and helps reviewers quickly understand changes."

O: How would you handle a merge conflict?

WHAT I LEARNED » "First, I'd run **git status** to see all conflicting files. I'd open each file - Git marks conflicts with <<<<<<< HEAD and >>>>>>> markers. I'd decide which version to keep (or combine both), remove the markers, and run **git add** to mark resolved. After all conflicts are fixed, **git commit** completes the merge. If I'm unsure, I'd ask the original author of the conflicting code rather than guess."

O: What's the difference between git pull and git fetch?

WHAT I LEARNED » "**git fetch** downloads commits from the remote but doesn't apply them to your local branch. **git pull** = **git fetch** + **git merge**. So **fetch** is safer - you can review before merging. I use **fetch** when I want to inspect what others did before integrating their work."

O: How do you handle secrets in Git?

WHAT I LEARNED » "Secrets NEVER go in Git. I use environment variables and **.gitignore**. For Spring Boot, I keep **application-secret.yml** ignored and inject values via env vars. For deployment, I use AWS Secrets Manager or similar. If I ever accidentally commit a secret, I immediately rotate it - removing from Git history isn't enough because bots scrape public repos within seconds."

O: Have you ever messed up Git? What happened?

WHAT I LEARNED » "Yes - early in MediBook I accidentally had a nested **.git** folder inside **src/**. When I ran **git add .**, Git treated **src/** as a submodule and would have pushed an empty folder to GitHub. I caught it because of the "embedded git repository" warning. I used **git rm -rf --cached** to unstage, deleted the inner **.git**, then re-staged. Lesson learned: always investigate warnings, never ignore them."

45. Glossary of Git Terms

- Branch** — A pointer to a series of commits. Like a parallel universe for code.
- Checkout** — Switch to a different branch or restore files to a previous state.
- Cherry-pick** — Copy a specific commit from one branch to another.
- Clone** — Download a complete copy of a remote repository.
- Commit** — A snapshot of your project at a point in time, with message and author.
- Commit hash** — Unique SHA-1 identifier for a commit (e.g., c5cd306).
- Conflict** — When Git can't auto-merge changes; requires manual resolution.
- Detached HEAD** — When you check out a specific commit instead of a branch tip.
- Diff** — The differences between two versions of files.
- Fetch** — Download remote changes without merging into local branch.
- Fork** — GitHub feature - create your own copy of someone else's repo.
- .git folder** — Hidden folder containing all Git's data (history, config, refs).
- .gitignore** — File listing patterns of files Git should ignore.
- HEAD** — Pointer to the current commit you're working on.
- Index (staging)** — Area where changes wait to be committed.
- Merge** — Combine two branches into one.
- Merge conflict** — Manual resolution needed when Git can't auto-combine changes.
- Origin** — Conventional name for your primary remote (usually GitHub).
- PAT** — Personal Access Token - replaces password for GitHub API/Git auth.
- Pull** — Fetch + merge in one step. Updates local branch from remote.
- Pull Request (PR)** — GitHub feature - propose changes to be merged.
- Push** — Upload local commits to remote repository.
- Rebase** — Re-apply commits on top of another branch (alternative to merge).
- Reflog** — Log of HEAD movements - your last hope when things go wrong.
- Remote** — A version of your repo hosted elsewhere (e.g., GitHub).
- Repository (repo)** — A project tracked by Git - includes .git folder.
- Reset** — Move HEAD to a different commit, optionally changing index/files.
- Revert** — Create a new commit that undoes a previous one.
- SHA-1** — Cryptographic hash function used to identify Git objects.
- Squash** — Combine multiple commits into one (during rebase or merge).
- Stage** — Mark changes ready to be committed.
- Stash** — Temporarily save uncommitted changes.
- Submodule** — A Git repo nested inside another Git repo (link, not copy).
- Tag** — Named reference to a specific commit (often for releases).
- Tracking branch** — Local branch linked to a remote branch.
- Untracked** — Files in working dir that Git doesn't know about yet.
- Upstream** — Remote/branch your local branch tracks for pull/push.
- Working directory** — Your actual project files on disk.

YOUR GIT JOURNEY IS COMPLETE!

You now have:

Professional Git workflow knowledge

Real debugging experience (embedded git, PowerShell errors)

GitHub repository live online

Feature branch ready for Phase 2

Same workflow used by Google, Netflix, Amazon, Microsoft

Keep committing. Keep pushing. Keep learning.

Repository: github.com/VIJAYKUMAR-G-CHATTARAKI/MediBook

Branch: **feature/phase-2-doctor-module**

Status: **Ready for Phase 2!**